Week 1 Progress Report

Network Quality Monitoring System - Foundation, Data Model & Background Scheduler

Prepared by	Zohaib Akmal
Report date	29 October 2026 (updated after debugging and optimization)
Plan period (Week 1)	19 - 25 October 2026
Completed through	Week 2 Wednesday (28 - 29 October 2026)
Repository	github.com/L1S24BSSE0071/Network-Quality-Monitoring-System
Version tag & tests	week-1 tagged; 122 automated pytest tests passing

1. Summary

The Week 1 milestone and Week 2 (Monday - Wednesday) background-checking tasks have been completed, thoroughly debugged and optimized: monitoring targets can be managed through the REST API, checks can be triggered on demand or run automatically in the background at each target's configured interval, and every result is stored in SQLite with a UTC timestamp. All deliverables are committed, merged to **main**, tagged **week-1**, and verified by an automated suite of **122 pytest tests**.

2. Milestone

"A target can be added, one check can be run by hand and its result appears in the database." Status: **Met** (plus automated background scheduling per target interval).

3. Tasks against the plan

Planned task	Status	Output
Requirements and scope document	**Done**	docs/requirements.md: 22 functional requirements, 8 validation rules, 9 non-functional requirements, exclusions and authorization statement
Database schema and system architecture	**Done**	docs/design.md: ER diagram, architecture diagram, check-flow and state diagrams (Mermaid, rendered on GitHub)
Repository, virtual environment, FastAPI skeleton, SQLite	**Done**	GitHub repository; pinned requirements.txt & requirements-dev.txt; app/main.py, app/database.py, app/models.py
Target API (create, list, get, edit, delete) with input validation	**Done**	app/routers/targets.py, app/schemas.py, app/errors.py; interactive docs at /docs
HTTP check with timeout and error handling; store one result	**Done**	app/checker.py; POST /targets/{id}/check and GET /targets/{id}/results
Background scheduler & failure isolation (Week 2 Mon - Wed)	**Done**	app/scheduler.py: APScheduler BackgroundScheduler running each enabled HTTP target at its interval_seconds; live sync on target add/edit/delete
Automated test suite, debugging & optimization	**Done**	tests/ (test_api.py, test_checker.py, test_scheduler.py): 122 automated tests passing in ~5 s
README draft, review notes & progress report	**Done**	README.md; docs/week1-review.md; docs/week1-progress-report.pdf

4. Deliverables

Deliverable	Location in repository
Requirements and scope document	docs/requirements.md
ER, architecture, sequence and state diagrams	docs/design.md
Target, check and results REST API	app/routers/targets.py, app/schemas.py, app/errors.py
HTTP checker and database models	app/checker.py, app/database.py, app/models.py
Background scheduler	app/scheduler.py
Automated test suite (122 tests)	tests/conftest.py, tests/test_api.py, tests/test_checker.py, tests/test_scheduler.py
README and Week 1 review notes	README.md, docs/week1-review.md

5. Definition of done

Criterion	Result	Evidence
The API starts without errors	**Met**	Server starts cleanly; /health returns ok; verified from a fresh clone using README steps
Invalid data returns a clear error message	**Met**	422 response lists each offending field and a plain-English reason (section 7)
Stored check result matches the manual check	**Met**	Result read back from GET /targets/{id}/results matches POST /targets/{id}/check
Schema and architecture diagrams are in the repository	**Met**	docs/design.md (4 Mermaid diagrams verified on GitHub)
Work is committed and tagged week-1	**Met**	All feature and fix branches merged (--no-ff) into main; tag week-1 pushed

6. Implementation, debugging and optimization

Technology: Python, FastAPI, SQLAlchemy 2.1, SQLite (WAL mode), httpx and APScheduler. **Design:** modular separation across database, models, schemas, errors, checker, scheduler and routers; each feature and bug-fix round was developed on its own branch and merged into **main**.

Validation & security hardening: required non-blank name (up to 100 chars, no C0/DEL/C1 control characters); well-formed http:// or https:// URL (valid port 1-65535, no userinfo credentials, no spaces, Unicode whitespace or backslashes); Ping host name or IPv4/IPv6 address (no % zone IDs); full IDNA/Punycode validation rejecting numeric hosts/TLDs (e.g. 256.1.1.1, example.123), leading/trailing hyphens and leading combining marks; strict integer/boolean validation (booleans and nulls rejected for integers; thresholds bounded to 1-3,600,000 ms with warning_ms < critical_ms); unknown fields rejected (extra='forbid'); and 32-bit positive TargetId path validation.

Performance & concurrency optimizations: pre-built shared TLS context (_SSL_CONTEXT), trust_env=False (skipping Windows proxy registry lookups, 11.3x faster client setup), headers-only streaming via client.stream('GET', url), and a daemon worker thread with a 1 s grace deadline so hung OS DNS lookups cannot block checks. Database sessions release pooled connections before network I/O, serialize conflicting writes with BEGIN IMMEDIATE, use composite indexes (ix_check_results_target_checked, ix_events_target_occurred), and discard stale in-flight checks if a target is modified, disabled or deleted mid-check.

Test suite module	Coverage	Result
tests/test_api.py	Target CRUD, defaults, IDNA/IPv6/URL/port/credential validation, control chars, strict types, unknown fields, malformed/empty JSON, cascade delete, concurrent PATCH serialization	Passed
tests/test_checker.py	200 OK, redirects, 404/503, headers-only streaming, timeout, hung DNS deadline abandonment, redirect loop, DNS/TLS/refused error mapping, connection pool release, stale check rejection (404/409)	Passed
tests/test_scheduler.py	Immediate check on start/add, per-target intervals, skipping disabled/ping targets, live sync on edit/delete, no restart on rename/threshold edit, failure isolation, 8-thread concurrent edits	Passed

Total automated test suite: 122 passed in ~5 seconds (zero network access, isolated temporary SQLite DB).

7. Demonstration evidence

Add a target - `POST /targets`

```
{"name": "Example", "host_or_url": "https://example.com"}
```

Run a check - `POST /targets/1/check` (stored with UTC timestamp ending in Z)

```
{"id": 1, "target_id": 1, "checked_at": "2026-10-28T13:52:39Z",
 "is_up": true, "latency_ms": 171.76, "http_status": 200,
 "error_message": null}
```

Failed check (unresolvable host) is stored without crashing

```
{"is_up": false, "latency_ms": null, "http_status": null,
 "error_message": "DNS lookup failed (host name could not be resolved)"}
```

Invalid input - warning threshold above critical threshold (status 422)

```
{"detail": [{"field": "(request)",
             "message": "warning_ms must be lower than critical_ms"}]}
```

8. Problems encountered and decisions

Problem or decision	Resolution
Week 1 finished three days after its planned end date because work started late.	Tasks were completed in plan order through Week 2 Wednesday, followed by comprehensive debugging and optimization.
Measured latency included TLS/CA setup (~690 ms) and Windows registry proxy lookups.	Pre-built a shared _SSL_CONTEXT, set trust_env=False (11.3x faster init), and streamed headers only; local checks now read ~3-4 ms.
Slow checks held SQLite connections and hung OS DNS lookups bypassed httpx timeouts.	Released DB transactions before network I/O, sized QueuePool to 20, and wrapped live checks in a daemon thread with a strict deadline.
HTTP status 400 or above & state-transition rules	Status >= 400 counts as down (keeping latency and http_status). Moving Critical -> Warning logs a warning event; returning to Normal logs recovered.

9. Remaining plan for Week 2 (Thursday - Sunday)

- Threshold evaluator: evaluate check outcomes against warning_ms and critical_ms, creating warning, critical and recovered events only on state changes.
- Uptime percentage calculation and API endpoints for history, events and target summaries.
- Unit tests for threshold state-transition logic, Week 2 review notes, and tagging week-2.
- Optional ping check only if time allows; per the plan, optional items are dropped first if the schedule is tight.